

Data Handling: Import, Cleaning and Visualisation

Solutions to Exercise/Workshop 2: Functions, iterations and data structures

Dr. Aurélien Sallin (Lecturer)

Ana Armendariz (Tutor)

1 October 2026

Table of contents

Exercise 1: warm-up on vectors	1
Exercise 2: warm-up on for-loops	2
Exercise 3: write a sum function	3
Exercise 4: robustness and warnings	3
Exercise 5: apply the function to a matrix	4
Exercise 6: slow and fast sloths	6
Exercise 7: data structures	7
Vectors, factors and lists	7
Matrices and arrays	8
Data frames and tibbles	9
Additional exercises for training	10
Exercise 8: standard deviation	10
Exercise 9: standard error	11
Exercise 10: t-test	11
Exercise 11: data frames, tibbles and data.tables	11

Exercise 1: warm-up on vectors

1. L after a number forces it to be stored as an integer instead of a double:

```
my_vector <- c(1L, 2L, 3L, 4L, 5L)
my_vector
```

```
[1] 1 2 3 4 5
```

2. There is no function to add a single value to an existing vector. Instead, you build a new vector that combines the old one with the new value, using `c()` again. `c()` combines its arguments into a new vector:

```
my_vector <- c(my_vector, 9L)
my_vector
```

```
[1] 1 2 3 4 5 9
```

3. `length()` counts the elements:

```
length(my_vector)
```

```
[1] 6
```

4. Square brackets index by position:

```
my_vector[3]
```

```
[1] 3
```

5. `%in%` checks membership, one value at a time:

```
9 %in% my_vector
```

```
[1] TRUE
```

```
10 %in% my_vector
```

```
[1] FALSE
```

Exercise 2: warm-up on for-loops

1. Loop over the vector and print each element:

```
colors <- c("red", "green", "blue")

for (color in colors) {
  print(color)
}
```

```
[1] "red"
```

```
[1] "green"
```

```
[1] "blue"
```

2. Compare each element to your favorite color inside the loop:

```
# Choose your favorite color:
favorite_color <- "green"

for (color in colors) {
  print(color == favorite_color)
}
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

Exercise 3: write a sum function

1. A function that adds up the elements of a vector with a `for` loop:

```
my_sum <- function(x) {  
  total <- 0          # start at zero  
  for (num in x) {    # visit every element of x  
    total <- total + num  
  }  
  return(total)  
}
```

2. Both give the same result, so the function works:

```
numbers <- c(1, 2, 3)  
my_sum(numbers)
```

```
[1] 6
```

```
sum(numbers)
```

```
[1] 6
```

Exercise 4: robustness and warnings

1. R stops with an error and computes nothing:

```
numbers2 <- c("1", "2", "3")  
my_sum(numbers2)
```

```
Error in `total + num`:  
! non-numeric argument to binary operator
```

2. `numbers2` does not contain numbers but text (a *character* vector). The quotation marks make the difference. You cannot add text with `+`.

```
str(numbers)
```

```
num [1:3] 1 2 3
```

```
str(numbers2)
```

```
chr [1:3] "1" "2" "3"
```

```
class(numbers)
```

```
[1] "numeric"
```

```
class(numbers2)
```

```
[1] "character"
```

3. An if statement checks the input before the loop runs. `is.numeric()` is TRUE for numeric and for integer vectors, so it is more flexible than `class(x) == "numeric"`.

If the input is wrong, the function returns NA, a missing value. It must not return 0: 0 looks like a valid sum, and whoever uses the result later would never notice that something went wrong.

```
my_sum <- function(x) {  
  if (!is.numeric(x)) {  
    warning("Wrong input! This function only accepts numeric or integer values!")  
    return(NA)  
  }  
  total <- 0  
  for (num in x) {  
    total <- total + num  
  }  
  return(total)  
}
```

4. The numeric vector works as before. The character vector now gives a warning and NA instead of an error:

```
my_sum(numbers)
```

```
[1] 6
```

```
my_sum(numbers2)
```

Warning in `my_sum(numbers2)`: Wrong input! This function only accepts numeric or integer values!

```
[1] NA
```

In practice you would not use your own function to add up numbers. `sum()` handles missing values (see `?sum`) and is much faster, because it is written in C rather than as an R loop:

```
big_vector <- rnorm(1e7) # ten million random numbers  
system.time(sum(big_vector))["elapsed"]
```

```
elapsed  
0.02
```

```
system.time(my_sum(big_vector))["elapsed"]
```

```
elapsed  
0.21
```

Exercise 5: apply the function to a matrix

1. A comma with a column index selects that column:

```
number_matrix <- matrix(1:9, nrow = 3)
number_matrix[, 1]
```

```
[1] 1 2 3
```

2. With a loop you create the container first and fill one slot per iteration.

```
sums_loop <- c()
for (i in 1:ncol(number_matrix)) {
  sums_loop[i] <- my_sum(number_matrix[, i])
}
sums_loop
```

```
[1] 6 15 24
```

3. `apply()` does the same in one line: `MARGIN = 2` applies the function over columns.

```
sums_apply <- apply(number_matrix, MARGIN = 2, my_sum)
sums_apply
```

```
[1] 6 15 24
```

4. `map_dbl()` from `purrr` iterates over the column indices and returns a numeric vector:

```
library(purrr)
```

Warning: package 'purrr' was built under R version 4.5.3

```
sums_map <- map_dbl(1:ncol(number_matrix), function(i) my_sum(number_matrix[, i]))
sums_map
```

```
[1] 6 15 24
```

5. When to use which:

- A `for` loop makes every step explicit. Use it when the iterations depend on each other (as in Exercise 6) or when you want full control. The logic is the same in every programming language.
- `apply()` is base R, shorter and less error-prone than a loop when the same function is applied independently to every row or column of a matrix.
- `map_dbl()` is the tidyverse equivalent. Its variants `map_int()`, `map_chr()`, `map()` return other types (an integer vector, a character vector, or a list) depending on what the function returns.

6. `?map` shows that `map()` “always returns a list”, while `map_dbl()` (like the other typed variants) checks that every result is a single double and simplifies the output to an atomic vector. `map()` is the general version: it works even if the function returns something more complex than a single number (a vector, a data frame, ...), for which no atomic vector could hold the results. The typed variants exist so you get a vector back when you know the output is homogeneous, without an extra `unlist()` step.

```
sums_map2 <- map(1:ncol(number_matrix), function(i) my_sum(number_matrix[, i]))
sums_map2
```

```
[[1]]
[1] 6

[[2]]
[1] 15

[[3]]
[1] 24
```

`sums_map2` is a list of three single-element vectors, printed one element per line with `[[1]]`, `[[2]]`, `[[3]]`, instead of the plain numeric vector `sums_map` from point 4.

Exercise 6: slow and fast sloths

1. and 2. Every year each population doubles (one offspring per sloth) and then loses its share of deaths. The loop runs as long as the slow sloths are at least as numerous as the fast ones. `year` counts which year we are at the beginning of, as in the table in the handout.

```
slow_sloths <- 1000
fast_sloths <- 1
slow_death_rate <- 0.40
fast_death_rate <- 0.30
year <- 1

while (slow_sloths >= fast_sloths) {
  slow_sloths <- slow_sloths * 2 * (1 - slow_death_rate)
  fast_sloths <- fast_sloths * 2 * (1 - fast_death_rate)
  year <- year + 1
}

cat("The fast sloths outnumber the slow sloths in year", year, "\n")
```

The fast sloths outnumber the slow sloths in year 46

```
cat("Slow sloths:", round(slow_sloths), " Fast sloths:", round(fast_sloths), "\n")
```

Slow sloths: 3657262 Fast sloths: 3764971

The populations grow by a factor of 1.2 (slow) and 1.4 (fast) per year, so we look for the smallest n with $1.4^n > 1000 \cdot 1.2^n$. That is $n = 45$ years of growth, and at the beginning of year 46 the fast sloths are ahead for the first time.

3. A `for` loop needs to know the number of iterations in advance. Here we do not know how many years it takes; we only know the condition under which we stop. That is what `while` is for. Note that a `while` loop runs forever if its condition never becomes `FALSE`. Here we are safe because the fast sloths grow faster every year.

Exercise 7: data structures

Note

This exercise also covers material from Lecture 4 (factors, lists, arrays, data frames and tibbles). Because exercise sessions are biweekly while lectures are weekly, this session falls between Lecture 3 and Lecture 4, so we work through Lecture 4 material before it is taught in class.

Vectors, factors and lists

1. Numbers are stored as *double* (double-precision floating point) by default, even if they look like integers.

```
student_ages <- c(18, 21, 20, 22, 19)
length(student_ages)
```

```
[1] 5
```

```
typeof(student_ages)
```

```
[1] "double"
```

2. A factor stores categorical data. Internally the values are integers with a label attached, the *levels*:

```
student_grades <- factor(c("A", "B", "A", "C", "B"))
student_grades
```

```
[1] A B A C B
Levels: A B C
```

```
levels(student_grades)
```

```
[1] "A" "B" "C"
```

Use a factor when a variable takes a fixed set of categories. R then knows that “A”, “B” and “C” are groups: `table()` and `summary()` count them, plots use them as groups, and regression functions such as `lm()` turn them into dummy variables automatically. Text would just be five strings.

3. A list can hold elements of different types and lengths. Elements are accessed by position with `[[ ]]` or by name with `$`:

```
student_names <- c("Anna", "Luca", "Sara", "Jonas", "Eva")

student_info <- list(names = student_names,
                    ages = student_ages,
                    grades = student_grades)

student_info[[1]]
```

```
[1] "Anna" "Luca" "Sara" "Jonas" "Eva"
```

```
student_info$names
```

```
[1] "Anna" "Luca" "Sara" "Jonas" "Eva"
```

```
student_info$grades
```

```
[1] A B A C B
```

```
Levels: A B C
```

4. A vector holds values of one type (all numbers, or all text). A factor is a vector for categorical data with a fixed set of levels. A list is needed when the elements are of different types: a vector would coerce everything to the most general type, here text.

```
c(student_names, student_ages) # the ages have become text
```

```
[1] "Anna" "Luca" "Sara" "Jonas" "Eva" "18" "21" "20" "22" "19"
```

Matrices and arrays

1. A matrix is a two-dimensional table of values of one type. Rows and columns can be named and used for indexing:

```
scores_matrix <- matrix(c(75, 82, 78,
                          88, 90, 92,
                          60, 65, 70), nrow = 3, byrow = TRUE)
rownames(scores_matrix) <- c("Student 1", "Student 2", "Student 3")
colnames(scores_matrix) <- c("Math", "English", "Science")

scores_matrix
```

	Math	English	Science
Student 1	75	82	78
Student 2	88	90	92
Student 3	60	65	70

```
scores_matrix["Student 2", "Science"]
```

```
[1] 92
```

2. An array adds a third dimension. Leaving an index empty selects everything along that dimension:

```
student_array <- array(
  c(75, 82, 78, 88, 90, 92, 60, 65, 70, # semester 1
    80, 84, 82, 91, 94, 93, 63, 68, 72), # semester 2
  dim = c(3, 3, 2),
  dimnames = list(c("Student 1", "Student 2", "Student 3"),
                  c("Math", "English", "Science"),
                  c("Semester 1", "Semester 2"))
)

student_array["Student 3", "Math", ]
```

Semester 1	Semester 2
78	82

Note that `array()` fills column by column, not row by row: the first three values are the Math column of semester 1.

3. A matrix has exactly two dimensions; an array can have any number. Use an array when the data has more than two dimensions, for example students by subjects by semesters, or pixels by rows by colour channels in an image. A matrix is a special case of an array.

Data frames and tibbles

1. A data frame is a table in which each column is a vector and different columns can have different types. Columns are accessed with `$`, rows with `[row, ]`:

```
students_df <- data.frame(  
  names = student_names,  
  ages = student_ages,  
  grades = student_grades,  
  attendance = c(85, 92, 78, 88, 90)  
)  
  
students_df$attendance
```

```
[1] 85 92 78 88 90
```

```
students_df[3, ]
```

	names	ages	grades	attendance
3	Sara	20	A	78

2. A matrix holds one type only: names and numbers together would all become text. A data frame keeps every column in its own type, which is what a data set needs.

A data frame is in fact a list of vectors of equal length, one per column. A list is more general: its elements can be anything and have any length, but you lose the table structure (rows as observations, columns as variables).

```
str(students_df)
```

```
'data.frame':  5 obs. of  4 variables:  
 $ names      : chr  "Anna" "Luca" "Sara" "Jonas" ...  
 $ ages       : num  18 21 20 22 19  
 $ grades     : Factor w/ 3 levels "A","B","C": 1 2 1 3 2  
 $ attendance : num  85 92 78 88 90
```

3. A tibble is the tidyverse version of a data frame. It prints more compactly, shows the column types, never converts text to factors on its own, and warns when you access a column that does not exist:

```
library(tibble)
```

```
Warning: package 'tibble' was built under R version 4.5.3
```

```
students_tibble <- as_tibble(students_df)

students_df
```

	names	ages	grades	attendance
1	Anna	18	A	85
2	Luca	21	B	92
3	Sara	20	A	78
4	Jonas	22	C	88
5	Eva	19	B	90

```
students_tibble
```

```
# A tibble: 5 x 4
  names    ages grades attendance
  <chr>   <dbl> <fct>         <dbl>
1 Anna     18 A             85
2 Luca     21 B             92
3 Sara     20 A             78
4 Jonas    22 C             88
5 Eva      19 B             90
```

```
students_df$nonexistent
```

NULL

```
students_tibble$nonexistent
```

Warning: Unknown or uninitialised column: `nonexistent`.

NULL

Additional exercises for training

Exercise 8: standard deviation

1. `x - mean_x` subtracts the mean from every element at once, `^2` squares every element, `sum()` adds them up. No loop needed.

```
my_sd <- function(x) {
  n <- length(x)
  if (n <= 1) {
    warning("At least two values are needed to compute a standard deviation.")
    return(NA)
  }
  mean_x <- mean(x)
  sqrt(sum((x - mean_x)^2) / (n - 1))
}
```

2. Compare with `sd()` on a random vector. `set.seed()` makes the random numbers reproducible.

```
set.seed(2026)
test_vector <- rnorm(10000) # 10 000 draws from a normal distribution with mean 0 and sd 1
my_sd(test_vector)
```

```
[1] 1.001459
```

```
sd(test_vector)
```

```
[1] 1.001459
```

Exercise 9: standard error

The new function calls `my_sd()`; the input check is already done there.

```
my_se <- function(x) {
  my_sd(x) / sqrt(length(x))
}

my_se(test_vector)
```

```
[1] 0.01001459
```

Exercise 10: t-test

```
my_t_test <- function(x, mu_0) {
  (mean(x) - mu_0) / my_se(x)
}

my_t_test(test_vector, mu_0 = 0)
```

```
[1] 0.366914
```

```
t.test(test_vector, mu = 0)$statistic
```

```
      t
0.366914
```

The true mean of `test_vector` is 0, so the hypothesised value is correct and the t-statistic is small, well inside ± 1.96 .

Exercise 11: data frames, tibbles and data.tables

1. A `data.table` is a third variant of the data frame, made for speed on large data. Inside the square brackets, the first argument filters rows and the second selects or computes columns:

```
library(data.table)
```

Warning: package 'data.table' was built under R version 4.5.3

Attaching package: 'data.table'

The following object is masked from 'package:purrr':

transpose

```
students_dt <- as.data.table(students_df)
```

```
students_dt[, ages]
```

```
[1] 18 21 20 22 19
```

```
students_dt[grades == "A"]
```

	names	ages	grades	attendance
	<char>	<num>	<fctr>	<num>
1:	Anna	18	A	85
2:	Sara	20	A	78

2. The same three tasks in the three dialects:

```
library(dplyr)
```

Warning: package 'dplyr' was built under R version 4.5.3

```
# keep the students with attendance above 85
students_df[students_df$attendance > 85, ]
```

	names	ages	grades	attendance
2	Luca	21	B	92
4	Jonas	22	C	88
5	Eva	19	B	90

```
students_tibble %>% filter(attendance > 85)
```

```
# A tibble: 3 x 4
  names  ages grades attendance
  <chr> <dbl> <fct>         <dbl>
1 Luca    21 B           92
2 Jonas   22 C           88
3 Eva     19 B           90
```

```
students_dt[attendance > 85]
```

	names	ages	grades	attendance
	<char>	<num>	<fctr>	<num>
1:	Luca	21	B	92
2:	Jonas	22	C	88
3:	Eva	19	B	90

```
# mean age of the students with grade A or B
mean(students_df$ages[students_df$grades %in% c("A", "B")])
```

```
[1] 19.5
```

```
students_tibble %>% filter(grades %in% c("A", "B")) %>% summarise(mean_age = mean(ages))
```

```
# A tibble: 1 x 1
  mean_age
  <dbl>
1    19.5
```

```
students_dt[grades %in% c("A", "B"), mean(ages)]
```

```
[1] 19.5
```

```
# add a column grade_numeric: A = 4, B = 3, C = 2
points <- c(A = 4, B = 3, C = 2)

students_df$grade_numeric <- points[as.character(students_df$grades)]
students_tibble <- students_tibble %>% mutate(grade_numeric = points[as.character(grades)])
students_dt[, grade_numeric := points[as.character(grades)]]

students_dt
```

	names	ages	grades	attendance	grade_numeric
	<char>	<num>	<fctr>	<num>	<num>
1:	Anna	18	A	85	4
2:	Luca	21	B	92	3
3:	Sara	20	A	78	4
4:	Jonas	22	C	88	2
5:	Eva	19	B	90	3

Base R uses square brackets and \$. `dplyr` chains verbs (`filter()`, `summarise()`, `mutate()`) with the pipe and reads like a sentence. `data.table` packs everything into `[rows, columns]` and modifies in place with `:=`, which avoids copying large data. We use `dplyr` in this course.